

Doc. no. P0718R0
Date: 2017-06-19
Project: Programming Language C++
Audience: SG1
Library Evolution Working Group
Library Working Group
Reply to: Alisdair Meredith <ameredith1@bloomberg.net>

Revising `atomic_shared_ptr` for C++20

Table of Contents

- 0. [Revision History](#)
 - [Revision 0](#)
- 1. [Introduction](#)
- 2. [Stating the problem](#)
 - [Consistency of propagation traits](#)
- 3. [Propose Solution](#)
 - [Move Atomic Smart Pointers to the `<memory>` Header](#)
 - [Restore the `atomic<shared\_ptr<T>>` Partial Specialization](#)
 - [Deprecate the C++11 Atomic Interface for `shared\_ptr`](#)
 - [Fix Up Minor Wording Nits](#)
- 4. [Other Directions](#)
- 5. [Formal Wording](#)
- 6. [Acknowledgements](#)
- 7. [References](#)

Revision History

Revision 0

Original version of the paper for the 2017 pre-Toronto mailing.

1 Introduction

The Concurrency TS introduced two atomic smart pointer classes, `atomic_shared_ptr` and `atomic_weak_ptr`, as a superior alternative to the atomic access API for `shared_ptr` in the C++ standard. This paper highlights several issues with that specification that should be resolved before merging its contents into a future C++ standard.

2 Stating the problem

The C++ standard provides an API to access and manipulate specific `shared_ptr` objects atomically, i.e., without introducing data races when the same object is manipulated from multiple threads without further synchronization. This API is fragile and error-prone, as `shared_ptr` objects manipulated through this API are indistinguishable from other `shared_ptr` objects, yet subject to the restriction that they may be manipulated/accessed *only* through this API. In particular, you cannot dereference such a `shared_ptr` without first loading it into another `shared_ptr` object, and then dereferencing through the second object.

The Concurrency TS addresses this fragility with a class template to implicitly wrap a `shared_ptr` and guarantee to access it only through the atomic access API. It provides a similar wrapper for `weak_ptr`, although the standard does not provide a corresponding atomic access API for `weak_ptrs`.

The atomic wrapper classes are placed in the `<atomic>` header, which is part of the free-standing library, while the smart pointer classes it wraps, and the corresponding atomic access API, are located in the `<memory>` header, which is not free-standing.

The atomic wrapper classes are designed to look like the named type aliases for specific instantiations of the atomic template, but are distinct class templates in their own right. While this risks confusing users, the specification is in terms of the primary atomic template, which may lead to subtle wording issues where the name of the template is involved. It also means that the smart pointer wrapper classes are not compatible with other functions in the `<atomic>` header that act on the atomic template, such as (C compatible) free-function APIs that correspond to member functions.

3 Propose Solution

3.1 Move Atomic Smart Pointers to the `<memory>` Header

The whole feature of wrapping smart pointers in an atomic type relies on contents of the `<memory>` header, including (as currently specified) the atomic access API to be invoked. Rather than create a dependency between the free-standing `<atomic>` header and the hosted `<memory>` header, we should simply move the whole smart-pointer specification into the `<memory>` header.

3.2 Restore the `atomic<shared_ptr<T>>` Partial Specialization

The smart pointer wrapper types in the Concurrency TS are deliberately named to look like a named alias of the atomic template. However, as they are not in fact such an alias, they are not usable in many APIs from the `<atomic>` header that are truly specified in terms of the atomic template. This is a recipe for confusion.

Rather than rename the atomic wrapper templates, this paper suggests revisiting the reason we chose to make such class templates in the first place, rather than more naturally providing partial specializations for the atomic class template. In particular, it appears that the reason we deferred on the original proposal ([N4058](#)) specializing the atomic template was concern about the constraint that τ must be trivially copyable for `atomic<T>`. However, this is fundamentally a constraint on what the library is committing to support when provided an otherwise unknown type by a user, rather than a constraint on what the library itself can support with more knowledge of a specific type. In particular, it is not clear how to specify constraints for the primary template to support non-trivial types, while it is clear how to specify known special cases such as smart pointers. Forcing the library to use a different interface will produce a substandard user experience, for no apparent gain.

This paper proposes restoring the original proposal's idea for `atomic<smart-pointer>` and supplying named alias templates, as per the atomic integral classes. We note in passing that there is no similar alias-template for:

```
template <typename T>
using atomic_pointer = atomic<T*>;
```

This would probably be a sought-for addition once the atomic wrappers have their specific names. However, it is not proposed in the initial draft of this paper.

3.3 Deprecate the C++11 Atomic Interface for `shared_ptr`

The atomic access API in clause 23 is fragile and easy to misuse. It overloads the atomic API for atomic objects in the `<atomic>` header with identical names and signatures for manipulating non-atomic objects. There is no other precedent for this (existing) API. It should be deprecated in favor of the new atomic smart pointer support. However, the atomic types in the Concurrency TS are specified in terms of that same API.

However, looking at the specification for the API in C++17, we see that it should be simple to rewrite the specification for the atomic pointer without reference to this API.

First, consumers of this API can access the smart pointer object only through this API. This would potentially be giving up an interoperability feature, but there is no public access to the wrapped smart pointer, so no other code can interact (directly) with our wrapped objects using this API. No generality is given up.

Secondly, each API documents a requirement that it is not called with null pointers, and this is not a useful precondition when replaced with a member-function based API, which can guarantee such preconditions are always satisfied without documenting them.

The important reason these functions are documented separately to those in clause 32 is to highlight that smart pointer objects have the same value when both the held pointer values are the same, and the shared control blocks are the same. This is not the same as the simple equality operator on these types, which compares only the held pointer value, and in fact, `weak_ptr` does not even offer an `operator==` to test. Similarly, the exchange semantics are specified as-if calling the member swap function.

Finally, there is no such atomic access API for `weak_ptr` at all, and it is simpler to directly specify the operations of the wrapper template than to provide a second fragile API to use by reference.

3.4 Fix Up Minor Wording Nits

There are a small number of minor nits in the specification that should also be cleaned up.

The Concurrency TS seems to assume that it is sufficient to state that a smart pointer is empty, assuming that means it is also null. However, through the aliasing constructor, `shared_ptr` objects can still hold valid pointers while empty, i.e., when not owning an object.

The Concurrency TS also assumes there is a "valid-but-unspecified" moved-from state for shared and weak pointers, but the specification for these types is precise in all cases, and objects are left empty after a successful move operation. It further gives the guarantee that lvalue references are not accessed again after the atomic step, but for no obvious reason does not give the same guarantee in the case of rvalue references. Given the state must be reset to empty (by shared/weak-pointer semantics) as part of the atomic update, there is no reason to not give the same guarantee for all reference types.

The Concurrency TS specification is missing the `value_type` type alias, and the static constant member `is_always_lock_free`.

The Concurrency TS types do not return a value from `operator=`, unlike the specification for `atomic` that they are defined in terms of. It would be very easy to non-atomically return the supplied argument after atomically storing the value, matching the primary `atomic` template, but the author prefers to keep the specification as close to the TS as possible, despite his own preference for preserving the primary template interfaces as much as possible.

4 Other Directions

The primary `atomic` template has a `volatile`-qualified overload for every member function. This allows for the idiom that objects that may be modified outside the current thread are marked as `volatile` to raise their visibility. The original proposal, as adopted by the TS and repeated in this paper, is to not support that for `atomic` smart pointers. Without casting prejudice on the idiom, non-atomic smart pointers do not have the overloaded constructors taking `volatile` references, so the idiom is not supported by the underlying types. This is one way that fundamental types typically differ from user-defined types, and the clause 32 specializations of `atomic` are dealing only with fundamental types that transparently handle the `volatile` qualifier.

The TS specification splits `compare_exchange` methods taking a non-atomic argument by value into two overloads for lvalues and rvalues, allowing for the most efficient argument passing for smart pointers. It does not do the same for other members that would similarly benefit, such as the by-value constructor and the assignment operator. This paper conservatively does not propose such a split either, while observing the original specification is in terms of a free-function API that is similarly specified to take `shared_ptr` objects by value, so there would be no benefit to making that split in the original specification. While the split seems beneficial once freed from that specification, the author has no implementation experience to confirm that it would be implementable or valuable.

As noted above, the partial specializations for `atomic` smart pointers could better respect the primary template interface, such as by (non-atomically) returning a value from the assignment operator.

5 Formal Wording

Make the following changes to the specified working paper. Note that as the proposed wording is not yet present in the C++ Working Paper, and there are no outstanding issues filed against these clauses of the Concurrency TS, we do not feel bound by the existing stable names for the clauses, and so propose new stable names appropriate for landing in the current C++ Working Paper.

Update the synopsis for the `<memory>` header as follows. Note that the addition of `reinterpret_pointer_cast` is an editorial drive-by, the function is already part of C++17.

23.10.2 Header `<memory>` synopsis [memory.syn]

1. The header `<memory>` defines several types and function templates that describe properties of pointers and pointer-like types, manage memory for containers and other template types, destroy objects, and construct multiple objects in uninitialized memory buffers (23.10.3–23.10.10). The header also defines the templates

unique_ptr, shared_ptr, weak_ptr, and various function templates that operate on objects of these types (23.11).

```
#include <atomic>
```

```
namespace std {
// ...

// 23.11.2.2, class template shared_ptr
template<class T> class shared_ptr;

// 23.11.2.2.6, shared_ptr creation
template<class T, class... Args>
    shared_ptr<T> make_shared(Args&&... args);
template<class T, class A, class... Args>
    shared_ptr<T> allocate_shared(const A& a, Args&&... args);

// 23.11.2.2.7, shared_ptr comparisons
template<class T, class U>
    bool operator==(const shared_ptr<T>& a, const shared_ptr<U>& b) noexcept;
template<class T, class U>
    bool operator!=(const shared_ptr<T>& a, const shared_ptr<U>& b) noexcept;
template<class T, class U>
    bool operator<(const shared_ptr<T>& a, const shared_ptr<U>& b) noexcept;
template<class T, class U>
    bool operator>(const shared_ptr<T>& a, const shared_ptr<U>& b) noexcept;
template<class T, class U>
    bool operator<=(const shared_ptr<T>& a, const shared_ptr<U>& b) noexcept;
template<class T, class U>
    bool operator>=(const shared_ptr<T>& a, const shared_ptr<U>& b) noexcept;

template <class T>
    bool operator==(const shared_ptr<T>& x, nullptr_t) noexcept;
template <class T>
    bool operator==(nullptr_t, const shared_ptr<T>& y) noexcept;
template <class T>
    bool operator!=(const shared_ptr<T>& x, nullptr_t) noexcept;
template <class T>
    bool operator!=(nullptr_t, const shared_ptr<T>& y) noexcept;
template <class T>
    bool operator<(const shared_ptr<T>& x, nullptr_t) noexcept;
template <class T>
    bool operator<(nullptr_t, const shared_ptr<T>& y) noexcept;
template <class T>
    bool operator<=(const shared_ptr<T>& x, nullptr_t) noexcept;
template <class T>
    bool operator<=(nullptr_t, const shared_ptr<T>& y) noexcept;
template <class T>
    bool operator>(const shared_ptr<T>& x, nullptr_t) noexcept;
template <class T>
    bool operator>(nullptr_t, const shared_ptr<T>& y) noexcept;
template <class T>
    bool operator>=(const shared_ptr<T>& x, nullptr_t) noexcept;
template <class T>
    bool operator>=(nullptr_t, const shared_ptr<T>& y) noexcept;

// 23.11.2.2.8, shared_ptr specialized algorithms
template <class T>
    void swap(shared_ptr<T>& a, shared_ptr<T>& b) noexcept;

// 23.11.2.2.9, shared_ptr casts
template<class T, class U>
    shared_ptr<T> static_pointer_cast(const shared_ptr<U>& r) noexcept;
template<class T, class U>
    shared_ptr<T> dynamic_pointer_cast(const shared_ptr<U>& r) noexcept;
template<class T, class U>
    shared_ptr<T> const_pointer_cast(const shared_ptr<U>& r) noexcept;
    template<class T, class U>
        shared_ptr<T> reinterpret_pointer_cast(const shared_ptr<U>& r) noexcept;

// 23.11.2.2.10, shared_ptr get_deleter
template<class D, class T>
    D* get_deleter(const shared_ptr<T>& p) noexcept;
```

```

// 23.11.2.2.11, shared_ptr I/O
template<class E, class T, class Y>
    basic_ostream& operator<< (basic_ostream<E, T>& os, const shared_ptr<Y>& p);

// 23.11.2.3, class template weak_ptr
template<class T> class weak_ptr;

// 23.11.2.3.6, weak_ptr specialized algorithms
template<class T> void swap(weak_ptr<T>& a, weak_ptr<T>& b) noexcept;

// 23.11.2.4, class template owner_less
template<class T = void> struct owner_less;

// 23.11.2.5, class template enable_shared_from_this
template<class T> class enable_shared_from_this;

// 23.11.2.6, shared_ptr atomic access
template <class T>
    bool atomic_is_lock_free(const shared_ptr<T>* p);

template <class T>
    shared_ptr<T> atomic_load(const shared_ptr<T>* p);
template <class T>
    shared_ptr<T> atomic_load_explicit(const shared_ptr<T>* p, memory_order mo);

template <class T>
    void atomic_store(shared_ptr<T>* p, shared_ptr<T> r);
template <class T>
    void atomic_store_explicit(shared_ptr<T>* p, shared_ptr<T> r, memory_order mo);

template <class T>
    shared_ptr<T> atomic_exchange(shared_ptr<T>* p, shared_ptr<T> r);
template <class T>
    shared_ptr<T> atomic_exchange_explicit(shared_ptr<T>* p, shared_ptr<T> r, memory_order mo);

template <class T>
    bool atomic_compare_exchange_weak(
        shared_ptr<T>* p, shared_ptr<T>* v, shared_ptr<T> w);
template <class T>
    bool atomic_compare_exchange_strong(
        shared_ptr<T>* p, shared_ptr<T>* v, shared_ptr<T> w);
template <class T>
    bool atomic_compare_exchange_weak_explicit(
        shared_ptr<T>* p, shared_ptr<T>* v, shared_ptr<T> w,
        memory_order success, memory_order failure);
template <class T>
    bool atomic_compare_exchange_strong_explicit(
        shared_ptr<T>* p, shared_ptr<T>* v, shared_ptr<T> w,
        memory_order success, memory_order failure);

// 23.11.2.7, hash support
template <class T> struct hash;
template <class T, class D> struct hash<unique_ptr<T, D>>;
template <class T> struct hash<shared_ptr<T>>;

// 23.11.3, atomic smart pointers
template template <class T> struct atomic<shared_ptr<T>>;
template template <class T> struct atomic<weak_ptr<T>>;

template template <class T> using atomic_shared_ptr = atomic<shared_ptr<T>>;
template template <class T> using atomic_weak_ptr = atomic<weak_ptr<T>>;

// ...
}

```

Add a new clause 23.11.3 to specify atomic smart pointers:

23.11.3 atomic specializations for smart pointers [util.smartptr.atomic]

1. The library shall provide partial specializations of the atomic template shared-ownership smart pointers. The behavior of all operations is as specified in §32.6 [atomics.types.generic], unless specified otherwise. The template parameter T of these partial specializations may be an incomplete type.

2. All changes to an atomic smart pointer in this subclause, and all associated use count increments, are guaranteed to be performed atomically. Associated use count decrements shall be sequenced after the atomic operation, but are not required to be part of it. Any associated deletion and deallocation are sequenced after the atomic update step and shall not be part of the atomic operation. [*Note: If the atomic operation uses locks, locks acquired by the implementation shall be held when any use count adjustments are performed, and shall not be held when any destruction or deallocation resulting from this is performed. — end note*]
3. For the purpose of compare exchange operations, two weak_ptr objects are equivalent if they are both empty, or if they shared ownership and store the same pointer value.

23.11.3.1 atomic specialization for shared_ptr [util.smartptr.atomic.shared]

```
namespace std {
template struct atomic<shared_ptr<T>> {
    using value_type = shared_ptr<T>;
    static constexpr bool is_always_lock_free = implementation-defined;

    bool is_lock_free() const noexcept;
    void store(shared_ptr<T> desired, memory_order order = memory_order_seq_cst) noexcept;
    shared_ptr load(memory_order order = memory_order_seq_cst) const noexcept;
    operator shared_ptr<T>() const noexcept;

    shared_ptr<T> exchange(shared_ptr<T> desired, memory_order = memory_order_seq_cst) noexcept;

    bool compare_exchange_weak(shared_ptr<T>& expected, const shared_ptr<T>& desired,
        memory_order success, memory_order failure) noexcept;
    bool compare_exchange_weak(shared_ptr<T>& expected, shared_ptr<T>&& desired,
        memory_order success, memory_order failure) noexcept;
    bool compare_exchange_strong(shared_ptr<T>& expected, const shared_ptr<T>& desired,
        memory_order success, memory_order failure) noexcept;
    bool compare_exchange_strong(shared_ptr<T>& expected, shared_ptr<T>&& desired,
        memory_order success, memory_order failure) noexcept;

    bool compare_exchange_weak(shared_ptr<T>& expected, const shared_ptr<T>& desired,
        memory_order order = memory_order_seq_cst) noexcept;
    bool compare_exchange_weak(shared_ptr<T>& expected, shared_ptr<T>&& desired,
        memory_order order = memory_order_seq_cst) noexcept;
    bool compare_exchange_strong(shared_ptr<T>& expected, const shared_ptr<T>& desired,
        memory_order order = memory_order_seq_cst) noexcept;
    bool compare_exchange_strong(shared_ptr<T>& expected, shared_ptr<T>&& desired,
        memory_order order = memory_order_seq_cst) noexcept;

    constexpr atomic() noexcept = default;
    atomic(shared_ptr<T> desired) noexcept;
    atomic(const atomic&) = delete;
    void operator=(const atomic&) = delete;
    void operator=(shared_ptr<T> desired) noexcept;

private:
    shared_ptr<T> p; // exposition only
};
};

constexpr atomic() noexcept = default;
```

1. *Effects:* Initializes the atomic object to an empty null value.

```
atomic(shared_ptr<T> desired) noexcept;
```
2. *Effects:* Initializes the object with the value desired. Initialization is not an atomic operation (4.7). [*Note: It is possible to have an access to an atomic object A race with its construction, for example by communicating the address of the just-constructed object A to another thread via memory_order_relaxed operations on a suitable atomic pointer variable, and then immediately accessing A in the receiving thread. This results in undefined behavior. — end note*]

```
void store(shared_ptr<T> desired, memory_order order = memory_order_seq_cst) noexcept;
```

3. *Requires:* The order argument shall not be memory_order_consume, memory_order_acquire, nor memory_order_acq_rel.

4. Effects: Atomically replaces the value pointed to by this with the value of desired as if by `p->swap(desired)`. Memory is affected according to the value of order.

```
void operator=(shared_ptr<T> desired) noexcept;
```

5. Effects: Equivalent to: `store(desired)`.

```
shared_ptr load(memory_order order = memory_order_seq_cst) const noexcept;
```

6. Requires: order shall not be `memory_order_release` or `memory_order_acq_rel`.

7. Effects: Memory is affected according to the value of order.

8. Returns: Atomically returns `*p`.

```
operator shared_ptr<T>() const noexcept;
```

9. Effects: Equivalent to: `return load();`.

```
shared_ptr<T> exchange(shared_ptr<T> desired, memory_order = memory_order_seq_cst) noexcept;
```

10. Effects: Atomically replaces `p` with `desired` as if by `p->swap(desired)`. Memory is affected according to the value of order. This is an atomic read-modify-write operation (4.7.1 [intro.races]).

11. Returns: Atomically returns the value of `*p` immediately before the effects.

```
bool compare_exchange_weak(shared_ptr<T>& expected, const shared_ptr<T>& desired,  
    memory_order success, memory_order failure) noexcept;  
bool compare_exchange_weak(shared_ptr<T>& expected, shared_ptr<T>&& desired,  
    memory_order success, memory_order failure) noexcept;  
bool compare_exchange_strong(shared_ptr<T>& expected, const shared_ptr<T>& desired,  
    memory_order success, memory_order failure) noexcept;  
bool compare_exchange_strong(shared_ptr<T>& expected, shared_ptr<T>&& desired,  
    memory_order success, memory_order failure) noexcept;
```

12. Requires: failure shall not be `memory_order_release` nor `memory_order_acq_rel`.

13. Effects: If `p` is equivalent to `expected`, assigns `desired` to `p` and has synchronization semantics corresponding to the value of success, otherwise assigns `p` to `expected` and has synchronization semantics corresponding to the value of failure.

14. Returns: true if `p` was equivalent to `expected`, false otherwise.

15. Remarks: Two `shared_ptr` objects are equivalent if they store the same pointer value and either share ownership, or both are empty. The weak form may fail spuriously. See 32.6.1.

16. If the operation returns true, `expected` is not accessed after the atomic update and the operation is an atomic read-modify-write operation (4.7) on the memory pointed to by this. Otherwise, the operation is an atomic load operation on that memory, and `expected` is updated with the existing value read from the atomic object in the attempted atomic update. The use_count update corresponding to the write to `expected` is part of the atomic operation. The write to `expected` itself is not required to be part of the atomic operation.

17.

18. If the desired value is passed by rvalue reference and the operation returns true, then `desired` is empty after the atomic update step.

```
bool compare_exchange_weak(shared_ptr<T>& expected, const shared_ptr<T>& desired,  
    memory_order order = memory_order_seq_cst) noexcept;
```

19. Effects: Equivalent to: `return compare_exchange_weak(expected, desired, order, fail_order);`; where `fail_order` is the same as `order` except that a value of `memory_order_acq_rel` shall be replaced by the value `memory_order_acquire` and a value of `memory_order_release` shall be replaced by the value `memory_order_relaxed`.

```
bool compare_exchange_weak(shared_ptr<T>& expected, shared_ptr<T>&& desired,  
    memory_order order = memory_order_seq_cst) noexcept;
```

20. Effects: Equivalent to: `return compare_exchange_weak(expected, std::move(desired), order, fail_order);`; where `fail_order` is the same as `order` except that a value of `memory_order_acq_rel` shall be replaced by the value `memory_order_acquire` and a value of `memory_order_release` shall be replaced by the value `memory_order_relaxed`.

```
bool compare_exchange_strong(shared_ptr<T>& expected, const shared_ptr<T>& desired,  
    memory_order order = memory_order_seq_cst) noexcept;
```


21. Effects: Equivalent to: `return compare_exchange_strong(expected, desired, order, fail_order);` where `fail_order` is the same as `order` except that a value of `memory_order_acq_rel` shall be replaced by the value `memory_order_acquire` and a value of `memory_order_release` shall be replaced by the value `memory_order_relaxed`.

```
bool compare_exchange_strong(shared_ptr<T>& expected, shared_ptr<T>&& desired,
    memory_order order = memory_order_seq_cst) noexcept;
```

22. Effects: Equivalent to: `return compare_exchange_strong(expected, std::move(desired), order, fail_order);` where `fail_order` is the same as `order` except that a value of `memory_order_acq_rel` shall be replaced by the value `memory_order_acquire` and a value of `memory_order_release` shall be replaced by the value `memory_order_relaxed`.

23.11.3.2 atomic specialization for weak_ptr [util.smartptr.atomic.weak]

```
namespace std {
template struct atomic<weak_ptr<T>> {
    using value_type = weak_ptr<T>;
    static constexpr bool is_always_lock_free = implementation-defined;

    bool is_lock_free() const noexcept;
    void store(weak_ptr<T> desired, memory_order order = memory_order_seq_cst) noexcept;
    weak_ptr load(memory_order order = memory_order_seq_cst) const noexcept;
    operator weak_ptr<T>() const noexcept;

    weak_ptr<T> exchange(weak_ptr<T> desired, memory_order = memory_order_seq_cst) noexcept;

    bool compare_exchange_weak(weak_ptr<T>& expected, const weak_ptr<T>& desired,
        memory_order success, memory_order failure) noexcept;
    bool compare_exchange_weak(weak_ptr<T>& expected, weak_ptr<T>&& desired,
        memory_order success, memory_order failure) noexcept;
    bool compare_exchange_strong(weak_ptr<T>& expected, const weak_ptr<T>& desired,
        memory_order success, memory_order failure) noexcept;
    bool compare_exchange_strong(weak_ptr<T>& expected, weak_ptr<T>&& desired,
        memory_order success, memory_order failure) noexcept;

    bool compare_exchange_weak(weak_ptr<T>& expected, const weak_ptr<T>& desired,
        memory_order order = memory_order_seq_cst) noexcept;
    bool compare_exchange_weak(weak_ptr<T>& expected, weak_ptr<T>&& desired,
        memory_order order = memory_order_seq_cst) noexcept;
    bool compare_exchange_strong(weak_ptr<T>& expected, const weak_ptr<T>& desired,
        memory_order order = memory_order_seq_cst) noexcept;
    bool compare_exchange_strong(weak_ptr<T>& expected, weak_ptr<T>&& desired,
        memory_order order = memory_order_seq_cst) noexcept;

    constexpr atomic() noexcept = default;
    atomic(weak_ptr<T> desired) noexcept;
    atomic(const atomic&) = delete;
    void operator=(const atomic&) = delete;
    void operator=(weak_ptr<T> desired) noexcept;
};
};
```

```
constexpr atomic() noexcept = default;
```

1. Effects: Initializes the atomic object to an empty null value.

```
atomic(weak_ptr<T> desired) noexcept;
```

2. Effects: Initializes the object with the value `desired`. Initialization is not an atomic operation (4.7). [*Note:* It is possible to have an access to an atomic object A race with its construction, for example by communicating the address of the just-constructed object A to another thread via memory_order_relaxed operations on a suitable atomic pointer variable, and then immediately accessing A in the receiving thread. This results in undefined behavior. — *end note*]

```
void store(weak_ptr<T> desired, memory_order order = memory_order_seq_cst) noexcept;
```

3. Requires: The `order` argument shall not be `memory_order_consume`, `memory_order_acquire`, nor `memory_order_acq_rel`.

4. Effects: Atomically replaces the value pointed to by this with the value of desired as if by `p->swap(desired)`. Memory is affected according to the value of order.

```
void operator=(weak_ptr<T> desired) noexcept;
```

5. Effects: Equivalent to: `store(desired)`.

```
weak_ptr load(memory_order order = memory_order_seq_cst) const noexcept;
```

6. Requires: order shall not be `memory_order_release` or `memory_order_acq_rel`.

7. Effects: Memory is affected according to the value of order.

8. Returns: Atomically returns `*p`.

```
operator weak_ptr<T>() const noexcept;
```

9. Effects: Equivalent to: `return load();`

```
weak_ptr<T> exchange(weak_ptr<T> desired, memory_order = memory_order_seq_cst) noexcept;
```

10. Effects: Atomically replaces `p` with `desired` as if by `p->swap(desired)`. Memory is affected according to the value of order. This is an atomic read-modify-write operation (4.7.1 [intro.races]).

11. Returns: Atomically returns the value of `*p` immediately before the effects.

```
bool compare_exchange_weak(weak_ptr<T>& expected, const weak_ptr<T> desired,  
    memory_order success, memory_order failure) noexcept;  
bool compare_exchange_weak(weak_ptr<T>& expected, weak_ptr<T>&& desired,  
    memory_order success, memory_order failure) noexcept;  
bool compare_exchange_strong(weak_ptr<T>& expected, const weak_ptr<T> desired,  
    memory_order success, memory_order failure) noexcept;  
bool compare_exchange_strong(weak_ptr<T>& expected, weak_ptr<T>&& desired,  
    memory_order success, memory_order failure) noexcept;
```

12. Requires: failure shall not be `memory_order_release` nor `memory_order_acq_rel`.

13. Effects: If `p` is equivalent to `expected`, assigns `desired` to `p` and has synchronization semantics corresponding to the value of success, otherwise assigns `p` to `expected` and has synchronization semantics corresponding to the value of failure.

14. Returns: true if `p` was equivalent to `expected`, false otherwise.

15. Remarks: Two `weak_ptr` objects are equivalent if they store the same pointer value and either share ownership, or both are empty. The weak form may fail spuriously. See 32.6.1.

16. If the operation returns true, `expected` is not accessed after the atomic update and the operation is an atomic read-modify-write operation (4.7) on the memory pointed to by this. Otherwise, the operation is an atomic load operation on that memory, and `expected` is updated with the existing value read from the atomic object in the attempted atomic update. The use_count update corresponding to the write to `expected` is part of the atomic operation. The write to `expected` itself is not required to be part of the atomic operation.

17.

18. If the desired value is passed by rvalue reference and the operation returns true, then `desired` is empty after the atomic update step.

```
bool compare_exchange_weak(weak_ptr<T>& expected, const weak_ptr<T> desired,  
    memory_order order = memory_order_seq_cst) noexcept;
```

19. Effects: Equivalent to: `return compare_exchange_weak(expected, desired, order, fail_order);` where `fail_order` is the same as `order` except that a value of `memory_order_acq_rel` shall be replaced by the value `memory_order_acquire` and a value of `memory_order_release` shall be replaced by the value `memory_order_relaxed`.

```
bool compare_exchange_weak(weak_ptr<T>& expected, weak_ptr<T>&& desired,  
    memory_order order = memory_order_seq_cst) noexcept;
```

20. Effects: Equivalent to: `return compare_exchange_weak(expected, std::move(desired), order, fail_order);` where `fail_order` is the same as `order` except that a value of `memory_order_acq_rel` shall be replaced by the value `memory_order_acquire` and a value of `memory_order_release` shall be replaced by the value `memory_order_relaxed`.

```
bool compare_exchange_strong(weak_ptr<T>& expected, const weak_ptr<T> desired,  
    memory_order order = memory_order_seq_cst) noexcept;
```

21. *Effects:* Equivalent to: `return compare_exchange_strong(expected, desired, order, fail_order);` where `fail_order` is the same as `order` except that a value of `memory_order_acq_rel` shall be replaced by the value `memory_order_acquire` and a value of `memory_order_release` shall be replaced by the value `memory_order_relaxed`.

```
bool compare_exchange_strong(weak_ptr<T>& expected, weak_ptr<T>&& desired,
    memory_order order = memory_order_seq_cst) noexcept;
```

22. *Effects:* Equivalent to: `return compare_exchange_strong(expected, std::move(desired), order, fail_order);` where `fail_order` is the same as `order` except that a value of `memory_order_acq_rel` shall be replaced by the value `memory_order_acquire` and a value of `memory_order_release` shall be replaced by the value `memory_order_relaxed`.

Move the old atomic support for shared pointers into Annex D:

23.11.2.6 shared_ptr-atomic access [util.smartptr.shared.atomic]

1. Concurrent access to a `shared_ptr`-object from multiple threads does not introduce a data race if the access is done exclusively via the functions in this section and the instance is passed as their first argument.
2. The meaning of the arguments of type `memory_order` is explained in 32.4.

```
template<class T>
bool atomic_is_lock_free(const shared_ptr<T>* p);
```

3. *Requires:* `p` shall not be null.
4. *Returns:* `true` if atomic access to `*p` is lock-free, `false` otherwise.
5. *Throws:* Nothing.

```
template<class T>
shared_ptr atomic_load(const shared_ptr* p);
```

6. *Requires:* `p` shall not be null.
7. *Returns:* `atomic_load_explicit(p, memory_order_seq_cst)`.
8. *Throws:* Nothing.

```
template<class T>
shared_ptr atomic_load_explicit(const shared_ptr* p, memory_order mo);
```

9. *Requires:* `p` shall not be null.
10. *Requires:* `mo` shall not be `memory_order_release` or `memory_order_acq_rel`.
11. *Returns:* `*p`.
12. *Throws:* Nothing.

```
template<class T>
void atomic_store(shared_ptr* p, shared_ptr r);
```

13. *Requires:* `p` shall not be null.
14. *Effects:* As if by `atomic_store_explicit(p, r, memory_order_seq_cst)`.
15. *Throws:* Nothing.

```
template<class T>
void atomic_store_explicit(shared_ptr* p, shared_ptr r, memory_order mo);
```

16. *Requires:* `p` shall not be null.
17. *Requires:* `mo` shall not be `memory_order_acquire` or `memory_order_acq_rel`.
18. *Effects:* As if by `p->swap(r)`.
19. *Throws:* Nothing.

```
template<class T>
shared_ptr atomic_exchange(shared_ptr* p, shared_ptr r);
```

20. *Requires:* `p` shall not be null.
21. *Returns:* `atomic_exchange_explicit(p, r, memory_order_seq_cst)`.
22. *Throws:* Nothing.

```
template<class T>
shared_ptr atomic_exchange_explicit(shared_ptr* p, shared_ptr r, memory_order mo);
```

23. *Requires:* *p* shall not be null.
24. *Effects:* As if by *p->swap(r)*.
25. *Returns:* The previous value of **p*.
26. *Throws:* Nothing.

```
template<class T>
bool atomic_compare_exchange_weak(shared_ptr* p, shared_ptr* v, shared_ptr w);
```

27. *Returns:* `atomic_compare_exchange_strong_explicit(p, v, w, memory_order_seq_cst, memory_order_seq_cst)`.

```
template<class T>
bool atomic_compare_exchange_strong(shared_ptr* p, shared_ptr* v, shared_ptr w);
```

28. *Requires:* *p* shall not be null.
- 29.
30. *Throws:* Nothing.

```
template
bool atomic_compare_exchange_weak_explicit(
    shared_ptr* p, shared_ptr* v, shared_ptr w,
    memory_order success, memory_order failure);
template
bool atomic_compare_exchange_strong_explicit(
    shared_ptr* p, shared_ptr* v, shared_ptr w,
    memory_order success, memory_order failure);
```

31. *Requires:* *p* shall not be null and *v* shall not be null. The *failure* argument shall not be `memory_order_release` nor `memory_order_acq_rel`.
32. *Effects:* If **p* is equivalent to **v*, assigns *w* to **p* and has synchronization semantics corresponding to the value of *success*; otherwise assigns **p* to **v* and has synchronization semantics corresponding to the value of *failure*.
33. *Returns:* `true` if **p* was equivalent to **v*, `false` otherwise.
34. *Throws:* Nothing.
35. *Remarks:* Two `shared_ptr` objects are equivalent if they store the same pointer value and share ownership. The weak form may fail spuriously. See 32.6.1.

D.14.x shared_ptr atomic access [depr.util.smartptr.shared.atomic]

```
namespace std {
// D.14.x shared_ptr atomic access
template <class T>
bool atomic_is_lock_free(const shared_ptr<T>* p);

template <class T>
shared_ptr<T> atomic_load(const shared_ptr<T>* p);
template <class T>
shared_ptr<T> atomic_load_explicit(const shared_ptr<T>* p, memory_order mo);

template <class T>
void atomic_store(shared_ptr<T>* p, shared_ptr<T> r);
template <class T>
void atomic_store_explicit(shared_ptr<T>* p, shared_ptr<T> r, memory_order mo);

template <class T>
shared_ptr<T> atomic_exchange(shared_ptr<T>* p, shared_ptr<T> r);
template <class T>
shared_ptr<T> atomic_exchange_explicit(shared_ptr<T>* p, shared_ptr<T> r, memory_order mo);

template <class T>
bool atomic_compare_exchange_weak(
    shared_ptr<T>* p, shared_ptr<T>* v, shared_ptr<T> w);
template <class T>
bool atomic_compare_exchange_strong(
    shared_ptr<T>* p, shared_ptr<T>* v, shared_ptr<T> w);
template <class T>
```

```

bool atomic_compare_exchange_weak_explicit(
    shared_ptr<T>* p, shared_ptr<T>* v, shared_ptr<T> w,
    memory_order success, memory_order failure);
template <class T>
bool atomic_compare_exchange_strong_explicit(
    shared_ptr<T>* p, shared_ptr<T>* v, shared_ptr<T> w,
    memory_order success, memory_order failure);
}

```

1. Concurrent access to a shared_ptr object from multiple threads does not introduce a data race if the access is done exclusively via the functions in this section and the instance is passed as their first argument.
2. The meaning of the arguments of type memory_order is explained in 32.4.

```

template<class T>
bool atomic_is_lock_free(const shared_ptr<T>* p);

```

3. Requires: p shall not be null.
4. Returns: true if atomic access to *p is lock-free, false otherwise.
5. Throws: Nothing.

```

template<class T>
shared_ptr atomic_load(const shared_ptr* p);

```

6. Requires: p shall not be null.
7. Returns: atomic_load_explicit(p, memory_order_seq_cst).
8. Throws: Nothing.

```

template<class T>
shared_ptr atomic_load_explicit(const shared_ptr* p, memory_order mo);

```

9. Requires: p shall not be null.
10. Requires: mo shall not be memory_order_release or memory_order_acq_rel.
11. Returns: *p.
12. Throws: Nothing.

```

template<class T>
void atomic_store(shared_ptr* p, shared_ptr r);

```

13. Requires: p shall not be null.
14. Effects: As if by atomic_store_explicit(p, r, memory_order_seq_cst).
15. Throws: Nothing.

```

template<class T>
void atomic_store_explicit(shared_ptr* p, shared_ptr r, memory_order mo);

```

16. Requires: p shall not be null.
17. Requires: mo shall not be memory_order_acquire or memory_order_acq_rel.
18. Effects: As if by p->swap(r).
19. Throws: Nothing.

```

template<class T>
shared_ptr atomic_exchange(shared_ptr* p, shared_ptr r);

```

20. Requires: p shall not be null.
21. Returns: atomic_exchange_explicit(p, r, memory_order_seq_cst).
22. Throws: Nothing.

```

template<class T>
shared_ptr atomic_exchange_explicit(shared_ptr* p, shared_ptr r, memory_order mo);

```

23. Requires: p shall not be null.
24. Effects: As if by p->swap(r).
25. Returns: The previous value of *p.
26. Throws: Nothing.

```

template<class T>
bool atomic_compare_exchange_weak(shared_ptr* p, shared_ptr* v, shared_ptr w);

```

27. Returns: atomic_compare_exchange_strong_explicit(p, v, w, memory_order_seq_cst, memory_order_seq_cst).

```
template<class T>
bool atomic_compare_exchange_strong(shared_ptr* p, shared_ptr* v, shared_ptr w);
```

28. Requires: p shall not be null.

29.

30. Throws: Nothing.

```
template
bool atomic_compare_exchange_weak_explicit(
    shared_ptr* p, shared_ptr* v, shared_ptr w,
    memory_order success, memory_order failure);
template
bool atomic_compare_exchange_strong_explicit(
    shared_ptr* p, shared_ptr* v, shared_ptr w,
    memory_order success, memory_order failure);
```

31. Requires: p shall not be null and v shall not be null. The failure argument shall not be memory_order_release nor memory_order_acq_rel.

32. Effects: If *p is equivalent to *v, assigns w to *p and has synchronization semantics corresponding to the value of success, otherwise assigns *p to *v and has synchronization semantics corresponding to the value of failure.

33. Returns: true if *p was equivalent to *v, false otherwise.

34. Throws: Nothing.

35. Remarks: Two shared_ptr objects are equivalent if they store the same pointer value and share ownership. The weak form may fail spuriously. See 32.6.1.

6 Acknowledements

Thanks to Herb Sutter, not only for the original proposal that was adopted for the TS, but especially for being available on short notice to advise on the history and rationale of the feature for this paper. Thanks to Stephan T. Lavavej for a detailed review of an early draft of this paper that improved it in many ways.

7 References

- [LWG #2334](#) atomic's default constructor requires "uninitialized" state even for types with non-trivial default-constructor
- [LWG #2980](#) Cannot compare_exchange empty pointers
- [N4058](#), [N4162](#), [N4260](#) Atomic Smart Pointers, Herb Sutter
- [N4577](#) Technical Specification for C++ Extensions for Concurrency, Artur Laksberg